Assignment #6

Designing the Data Layer and Backend for a NoSQL Recommendation Platform

Prepared by: Dilnaz Zholmaganbet, **Group:** CSE-2601M

Checked by: Basheyeva Zhuldyz

Course: Advanced Databases

Project: MemoryAssist – memory recommendation backend for AR glasses

Goals and Objectives

Goal: to apply NoSQL database principles to a recommendation platform from my research area: design a data model for users, items and their interactions that keeps the common queries fast, and implement the core backend and API of the platform.

Objectives:

1. Choose a topic related to my dissertation and identify the users, items and interactions.
2. Choose a NoSQL database and write a project proposal with the data model, the planned recommendation algorithm and the section "Relevance to the dissertation".
3. Design the database schema, compare the trade-offs of different models and define the indexes the queries need.
4. Implement user registration, login and profiles.
5. Implement the item catalog with create, read, update and delete operations.
6. Record the history of user interactions with items.
7. Implement search by name, category and other attributes.
8. Provide an API for all of the above, fill the database with test data, and document the API and the setup.

Tools were used:

- MongoDB Community Server (local, port 27017) and MongoDB Compass – the database and its graphical client.
- Python 3 with FastAPI (web framework), Uvicorn (server), PyMongo (MongoDB driver), Pydantic (validation), bcrypt (password hashing) and PyJWT (login tokens).
- Swagger UI – the interactive API documentation that FastAPI generates at /docs. All API tests in this report were made there.
- Claude – AI assistant (Anthropic, 2026). I used it as a helper to plan the data model, write and test the code, and prepare the documents. I ran the project on my computer, checked the results and agreed the topic with my supervisor.

Deliverables:

Part I of this report is the project proposal, including the section "Relevance to the dissertation". Part II shows the implementation with screenshots. The source code is in the folder `memoryassist`; the API documentation is the file `API.md` together with the Swagger page at /docs; the setup and run instructions are in `README.md`.

Part I. Project Proposal

1. Introduction

Smart glasses with cameras and eye tracking, such as Meta's Project Aria glasses, can record almost everything a person sees. Storing everything is expensive and makes it hard to find the one moment that matters, for example where a person left their keys. Storing only what the person looked at is not enough either, because looking at an object does not guarantee that it will be remembered. My dissertation, "Cognitively Informed Memory Retrieval for AR Glasses", studies what people actually forget and uses this knowledge to decide what smart glasses should store and when they should remind the user.

This project builds the data layer and backend for that idea as a recommendation platform. The platform stores users, everyday items and the interactions between them: placing an item somewhere while the glasses record gaze, trying to recall the item the next day, and asking the system where the item is. In the Final Project the platform will recommend which items a user is most likely to forget, so that the glasses can keep those memories and show reminders for them. The working name of the platform is MemoryAssist.

2. Users, Items and Interactions

Users are the people who wear the glasses. In the planned experiment they are 25–30 Nazarbayev University students who walk along a corridor in Block 3, find three rooms and leave 12 small items wherever they like. A user profile stores only anonymous information: a participant code, age group, field of study, whether the person wears glasses, and a self-rating of memory from 1 to 5. There is also a researcher role, which manages the item catalog and sees the analytics.

Items are the everyday objects that are placed and later recalled: a key, a pen, a USB flash drive, a student ID card, earphones case, coin, sticky notes and others. Each item has a name, description and category, plus attributes that matter for memory: colour, size and salience (how noticeable the item is, from 1 to 5).

Interactions connect users and items. There are three types. A "placed" interaction records where the user left the item (a location on the corridor map) and the gaze features measured by the glasses: total dwell time, number of fixations and the number of times the user looked back at the item. A "recalled" interaction stores the result of the next-day memory test: whether the place was remembered, the distance between the marked and the real place, and the user's confidence. A "queried" interaction is created when the user asks the system where an item is. Locations (rooms and landmarks of the corridor with x/y coordinates) are stored as a separate small collection.

3. Choice of the NoSQL Database

I chose **MongoDB** as the main database. The data of this platform is a set of entities with flexible, nested fields: a placed interaction has gaze data, a recalled one has recall data, and new sensor features from the glasses can be added later without changing a schema. MongoDB stores such records naturally as JSON-like documents. It also gives everything the planned queries need: compound indexes for the history of a user, a text index for item search, a 2d index for positions on the corridor map, and the aggregation framework for statistics such as the forget rate of every item, which is the starting point for the recommender.

I compared it with two other NoSQL databases. **Redis** is an in-memory key-value store and is very fast, but it is designed for caching and simple structures, and complex queries over the history (filters, joins with items, grouping) would have to be written in the application. It could be added later as a cache for the most recent reminders. **Neo4j** is a graph database and is strong for recommendations of the type "users who forgot this item also forgot that one", because users, items and locations form a graph. However, most operations of this assignment are document operations (registration, catalog CRUD, logging events, text search), and the volume of interaction events grows fast, which suits a document store better. For the Final Project, Neo4j remains an option for graph-based recommendation, for example by exporting the user–item "forgot" relationships from MongoDB.

4. Data Modeling Approach

The database `memoryassist` has four collections: `users`, `items`, `locations` and `interactions`. The main design decision is how to store the interactions. One option is to embed the history inside each user document as an array. This makes reading the full history of one user a single read, but the array grows with every event, a document in MongoDB is limited to 16 MB, and statistics across all users (for example, how often each item is forgotten) would need to unwind many large arrays. The second option, which I chose, is a separate `interactions` collection where every event is a small document that references the user, the item and the location by their ids. Writes stay cheap, the collection can grow without limits, and the history of one user is still fast thanks to an index on `userId` and `timestamp`.

Small, stable data is embedded: the profile is embedded in the user document, the attributes in the item document, and the gaze and recall values in the interaction document, because they are always read together with their parent. The item name is not copied into interactions; the history endpoint adds it with a `$lookup`, so renaming an item never leaves old names in the history.

Example of an interaction document:

```
{
  "_id": ObjectId("..."),
  "userId": ObjectId("..."),
  "itemId": ObjectId("..."),
  "type": "placed",
  "sessionId": "S01",
  "locationId": ObjectId("..."),
  "timestamp": ISODate("2026-11-02T10:12:00Z"),
  "gaze": { "dwellMs": 1650, "fixations": 6, "revisits": 1 }
}
```

The indexes are chosen from the queries the API runs. On `users`, a unique index on `email` is used by login and prevents duplicate accounts. On `items`, a text index on `name` and `description` (with a higher weight for the name) is used by search, single-field indexes on `category` and `attributes.color` are used by the filters, and a unique index on `name` keeps the catalog clean. On `locations`, a 2d index on `position` finds the nearest landmarks to a point on the map. On `interactions`, the compound index `{userId, timestamp}` serves the history of a user sorted by time, `{userId, itemId, type, timestamp}` serves the question "where did I leave item X" with one index lookup, and `{itemId, type}` serves the statistics per item.

5. Backend and API

The backend is written in Python with FastAPI and the PyMongo driver. FastAPI validates every request with Pydantic models and generates interactive API documentation automatically. Passwords are stored as bcrypt hashes, and login returns a JSON Web Token that the client sends in the Authorization header. The API covers registration, login and profile editing; CRUD for the item catalog (only for the researcher role); recording and reading interactions; search by text, category, colour and size; the question "where did I leave my item"; and an analytics endpoint with the forget rate of every item. The database is filled with synthetic test data that imitates the planned experiment.

6. Planned Recommendation Algorithm (Final Project)

The goal of the recommender is to rank, for a given user, the items they are most likely to forget, so that the glasses keep those memories and remind the user about them first. I plan a hybrid approach with two parts.

The first part is a **forgetting-risk model** based on content and behaviour. For every placed item it uses the gaze features (dwell time, fixations, revisits), the item attributes (salience, size) and the user's profile (memory self-rating) to predict the probability that the item will be forgotten the next day. I will start with logistic regression, because its coefficients are easy to interpret and show which gaze features matter most, which is also a research question of the dissertation, and then compare it with a gradient-boosted tree model.

The second part is **item-based collaborative filtering**. From the recall results I can build a user–item matrix where a value shows whether the user forgot the item. Items that are often forgotten by the same people are similar, so if a user forgot the coin and the eraser, the system can predict a higher risk for items that similar users also forgot. This part helps for new items or when gaze data is missing or noisy.

The final score will be a weighted combination of the two parts, and the top-k items will be returned as recommendations. The evaluation will use the real next-day recall as ground truth: ROC AUC for the forgetting-risk model and precision@k and recall@k for the ranked list, compared with two simple baselines, "remind about everything" and "remind only about what the user looked at least". The analytics endpoint already implemented in this assignment calculates the forget rate per item and is the first step of this pipeline.

7. Data and Ethics

Real data from participants can be collected only after approval by the NU Institutional Research Ethics Committee (IREC). For this assignment I therefore use only synthetic data: 30 virtual participants, 12 items, 10 corridor locations and 780 interactions. The generator makes longer gaze and looking back at an item increase the chance of remembering it, so the data has a realistic pattern without containing any personal information. The real data, when collected, will be stored with participant codes instead of names, and the same data model will be used without changes.

8. Relevance to the Dissertation

My dissertation asks a simple question with a practical goal: can we learn what people actually forget and use it to decide what AR glasses should store? The experiment collects three kinds of data. The Aria glasses record where each participant looks while walking along the corridor and leaving 12 items. One day later, without the glasses, participants mark on a map where they left each item. The analysis then checks whether eye gaze predicts which items were forgotten and uses this to build and test a memory model. The MemoryAssist platform is built around exactly these three kinds of data, so it is not a separate exercise but the data infrastructure the dissertation needs.

First, the data model of this assignment is the storage design for the experiment. Users correspond to participants, items to the 12 objects, placed interactions to the gaze recordings, and recalled interactions to the next-day map test. Because the gaze features are stored as flexible sub-documents, the same model can hold the features we will extract from the Aria recordings, even if the final set of features changes during the research. The indexes and the history endpoint make it easy to pull all recordings of one participant or all results for one item, which is what the statistical analysis needs.

Second, the recommendation task of the Final Project is the memory model of the dissertation written as a recommender. Predicting "which items will this user forget" and ranking them is the same as deciding which memories the glasses should keep and which reminders they should

show. The forgetting-risk model will directly test the main hypothesis, that gaze features predict forgetting, and the collaborative part will show whether patterns from other people improve the prediction for a new person. The evaluation against the next-day recall is the same evaluation the dissertation will report.

Third, the backend makes the idea testable as a system and not only as a statistical model. The "where did I leave my item" endpoint is a simple version of the memory retrieval function that AR glasses would offer, and logging every query as an interaction will later let us measure how often people actually need help and for which items. In the dissertation, the platform can be described as the prototype of the storage and retrieval layer, and its results, such as the forget rate per item and the accuracy of the recommender, can be used as evidence for which information smart glasses should store. Building it now with synthetic data also lets me test the full pipeline before IREC approval, so that the real data can be analysed as soon as it is collected.

Part II. Implementation

9. Database Schema and Indexes

The schema below implements the approach described in Section 4: interactions are a separate collection that references users, items and locations, and small data that is always read with its parent is embedded.

users:

```
{ "_id": ObjectId, "email": "p01@memoryassist.test",
  "passwordHash": "$2b$12$...", "role": "participant" | "admin",
  "profile": { "displayName", "ageGroup", "fieldOfStudy", "wearsGlasses",
"memorySelfRating" },
  "createdAt": ISODate }
```

items:

```
{ "_id": ObjectId, "name": "Key", "description": "Small metal key on a red
ring",
  "category": "personal belongings",
  "attributes": { "color": "silver", "size": "small", "salience": 3 },
"createdAt": ISODate }
```

locations:

```
{ "_id": ObjectId, "code": "L03", "name": "Window sill by the stairs",
  "zone": "stairs", "position": [12.0, 2.8] }
```

interactions:

```
{ "_id": ObjectId, "userId": ObjectId, "itemId": ObjectId, "sessionId": "S01",
  "type": "placed" | "recalled" | "queried", "timestamp": ISODate,
  "locationId": ObjectId,                                  // placed
  "gaze": { "dwellMs": 1650, "fixations": 6, "revisits": 1 },   // placed
  "recall": { "correct": false, "errorMeters": 7.5, "confidence": 2 } }  //
recalled
```

The indexes are created by the code (`app/database.py`) every time the server starts. Each index exists because a specific query uses it:

```
users.create_index([("email", 1)], unique=True)                # login, no
duplicate accounts
items.create_index([("name", "text"), ("description", "text")],
                   weights={"name": 3, "description": 1})      # full-text
search
items.create_index([("category", 1)])                          # filter by
category
items.create_index([("attributes.color", 1)])                  # filter by
colour
items.create_index([("name", 1)], unique=True)                 # no duplicate
items
locations.create_index([("position", "2d")])                   # nearest
locations
interactions.create_index([("userId", 1), ("timestamp", -1)])  # history of a
user
interactions.create_index([("userId", 1), ("itemId", 1),
                           ("type", 1), ("timestamp", -1)])    # "where did I
leave X"
interactions.create_index([("itemId", 1), ("type", 1)])        # statistics
per item
```

The compound indexes follow the rule equality first, then sort: in the history query the user id is matched exactly and the results are sorted by time, so `userId` comes first and `timestamp` second. The query "where did I leave item X" matches user, item and type exactly and takes the newest event, so one index answers it without sorting in memory.

10. Project Setup and Test Data

The project is a Python package with the FastAPI application in `app/` and the seed script `seed.py`. `database.py` connects to MongoDB and creates the indexes, `security.py` hashes passwords and creates tokens, `schemas.py` validates the request data, and the folder `routers/` contains the endpoints for authentication, items and interactions.

```
memoryassist/
  app/main.py  app/database.py  app/security.py  app/schemas.py
  app/routers/auth.py  app/routers/items.py  app/routers/interactions.py
  seed.py  requirements.txt  README.md  API.md
```

MongoDB was started in one terminal window. In a second window I created a virtual environment, installed the libraries, loaded the test data and started the server:

Commands:

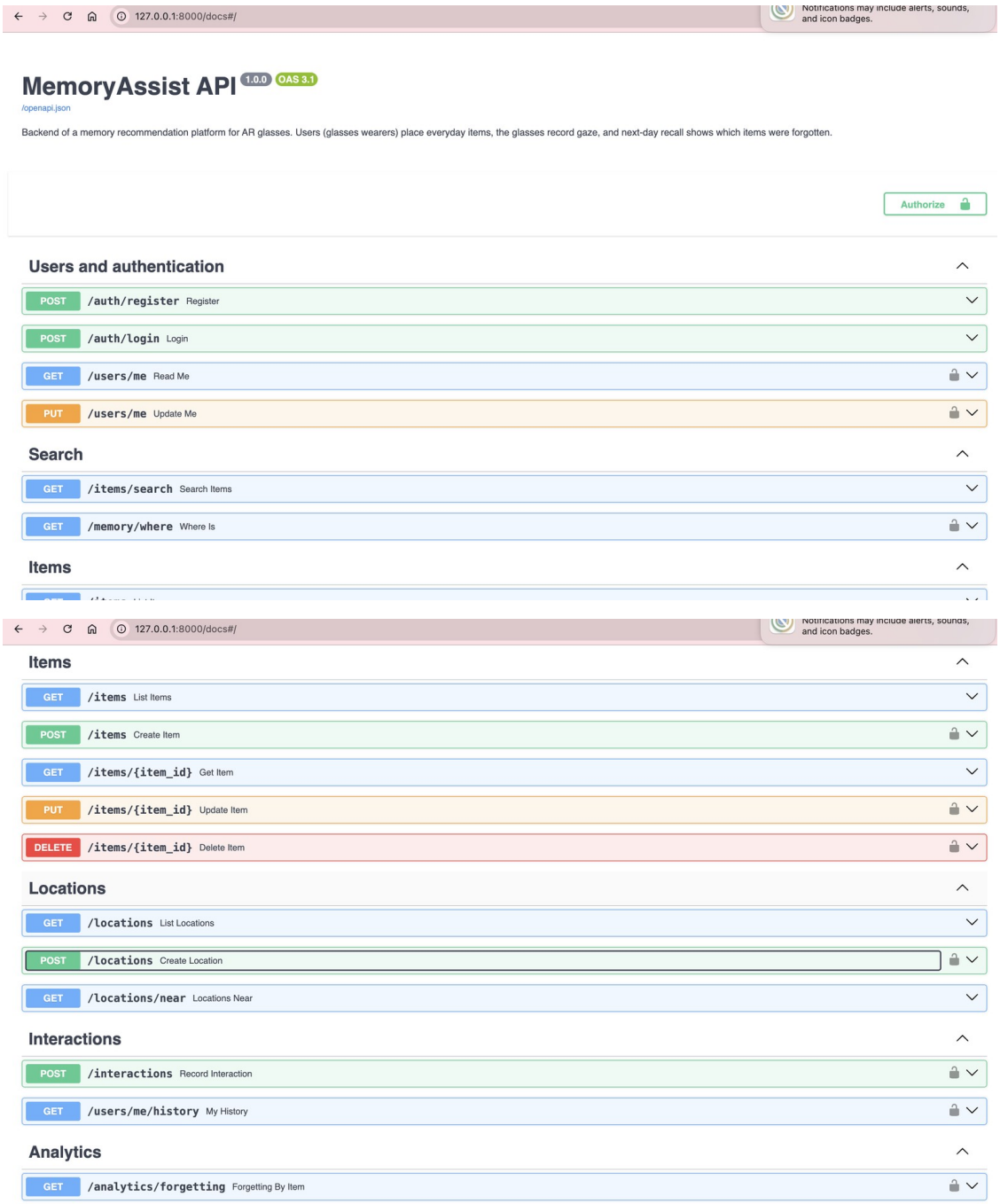
```
cd ~/Desktop/mongodb/memoryassist
python3 -m venv venv
source venv/bin/activate
pip install --isolated -r requirements.txt
python seed.py
uvicorn app.main:app --reload
```

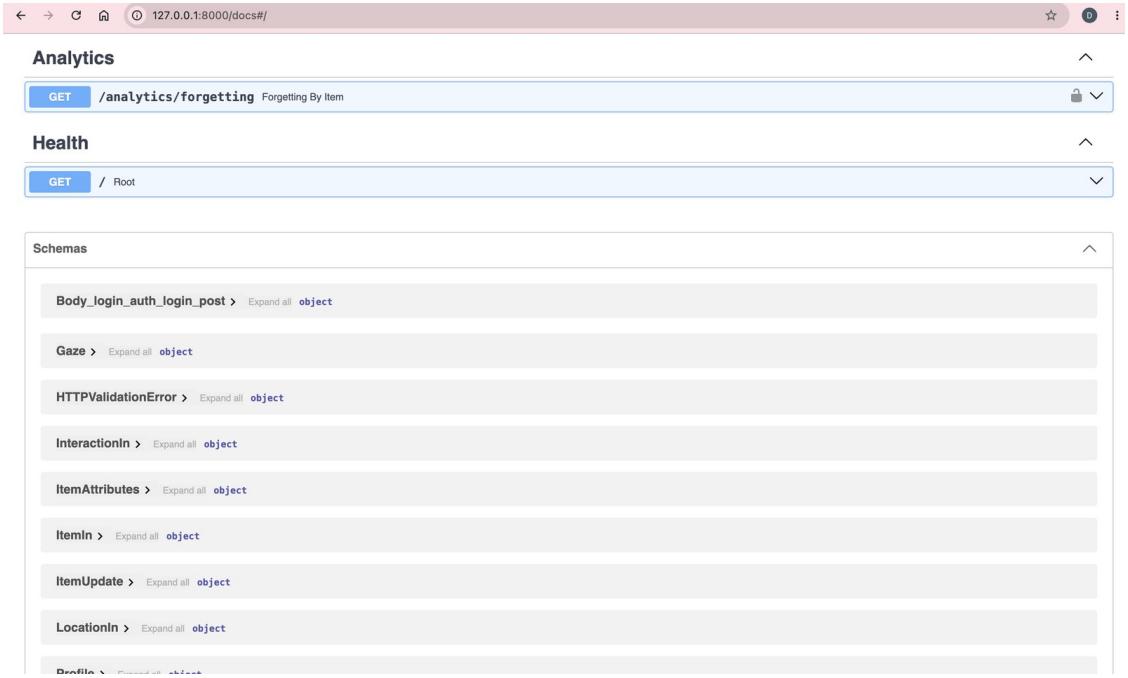


Result: the seed script created 31 users (1 researcher and 30 participants), 12 items, 10 corridor locations and 780 interactions (360 placed, 360 recalled, 60 queried), and the server started at http://127.0.0.1:8000. The generator makes longer gaze and looking back at an item increase the chance of remembering it, so about half of the items are remembered, and the remembered ones have a longer median dwell time. I used `--isolated` because my pip configuration had an extra package server that was not reachable.

11. API Overview

FastAPI generates the documentation page at http://127.0.0.1:8000/docs from the code. It lists all endpoints grouped by topic, shows which ones need a login (lock icon) and allows sending requests directly from the browser. The written description of every endpoint is in `API.md`.


127.0.0.1:8000/docs#/
Notifications may include alerts, sounds, and icon badges.
MemoryAssist API
1.0.0
OAS 3.1
/openapi.json
Backend of a memory recommendation platform for AR glasses. Users (glasses wearers) place everyday items, the glasses record gaze, and next-day recall shows which items were forgotten.
Authorize
Users and authentication
POST /auth/register Register
POST /auth/login Login
GET /users/me Read Me
PUT /users/me Update Me
Search
GET /items/search Search Items
GET /memory/where Where Is
Items
127.0.0.1:8000/docs#/
Notifications may include alerts, sounds, and icon badges.
Items
GET /items List Items
POST /items Create Item
GET /items/{item_id} Get Item
PUT /items/{item_id} Update Item
DELETE /items/{item_id} Delete Item
Locations
GET /locations List Locations
POST /locations Create Location
GET /locations/near Locations Near
Interactions
POST /interactions Record Interaction
GET /users/me/history My History
Analytics
GET /analytics/forgetting Forgetting By Item



Result: the API has 17 endpoints in six groups (users and authentication, search, items, locations, interactions and analytics) plus a health check at /. The Schemas section at the bottom shows the validation models of the request bodies.

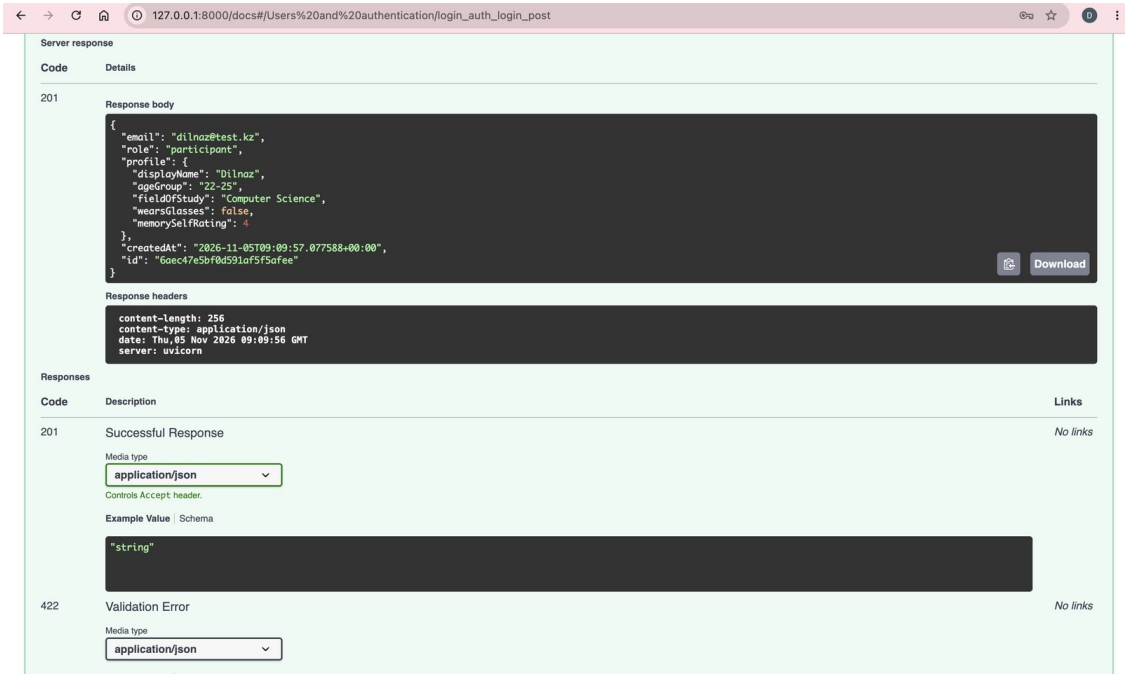
12. User Registration and Profiles

12.1 Registration:

POST /auth/register creates a participant account. The password is hashed with bcrypt before it is saved, and the response never contains it.

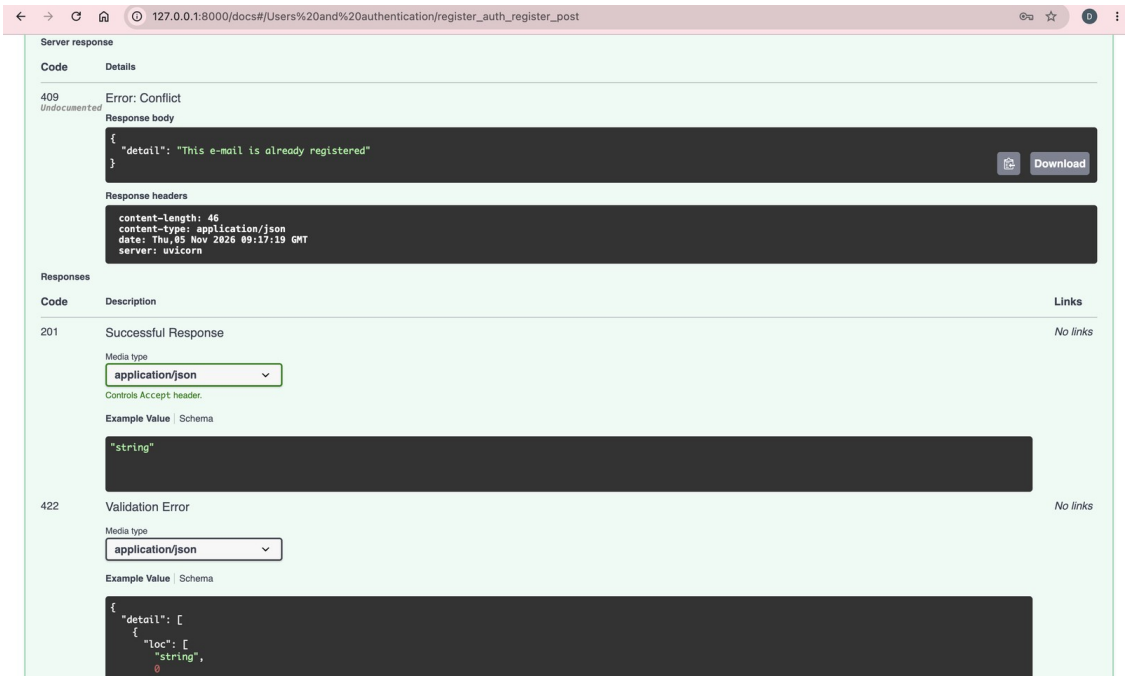
Request:

```
{
  "email": "dilnaz@test.kz",
  "password": "secret123",
  "profile": {"displayName": "Dilnaz", "ageGroup": "22-25", "fieldOfStudy":
"Computer Science",
              "wearsGlasses": false, "memorySelfRating": 4}
}
```



Result: 201 Created with the new user, its id and role "participant", without the password.

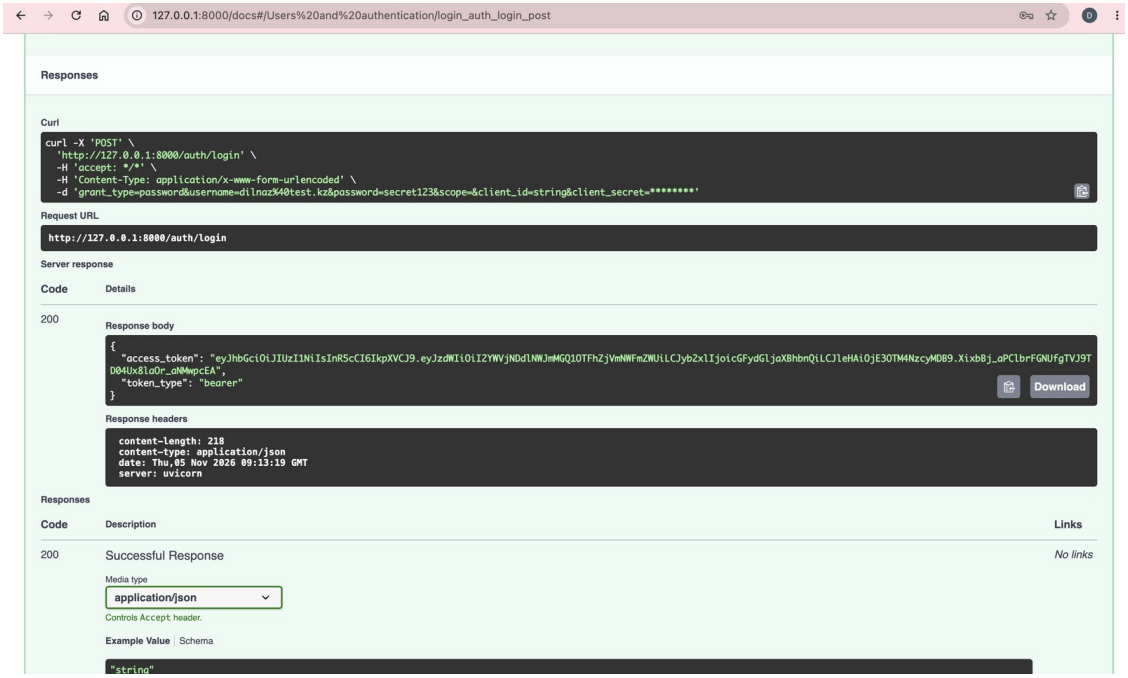
Sending the same request again:



Result: 409 Conflict, "This e-mail is already registered". The unique index on email rejects the second insert, and the API turns the database error into a clear message.

12.2 Login:

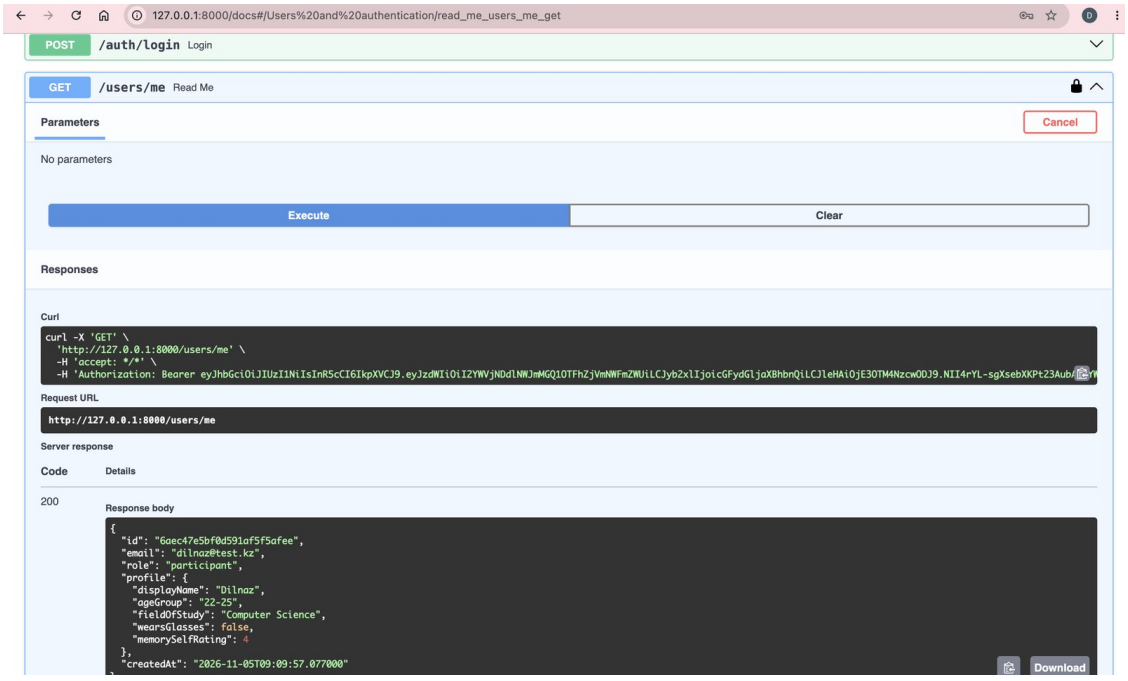
POST /auth/login takes the e-mail and password as a form and returns a JSON Web Token. I also used the Authorize button of the docs page, which logs in once and then sends the token with every protected request.



Result: 200 OK with access_token (valid for 120 minutes) and token_type "bearer".

12.3 Viewing and Editing the Profile:

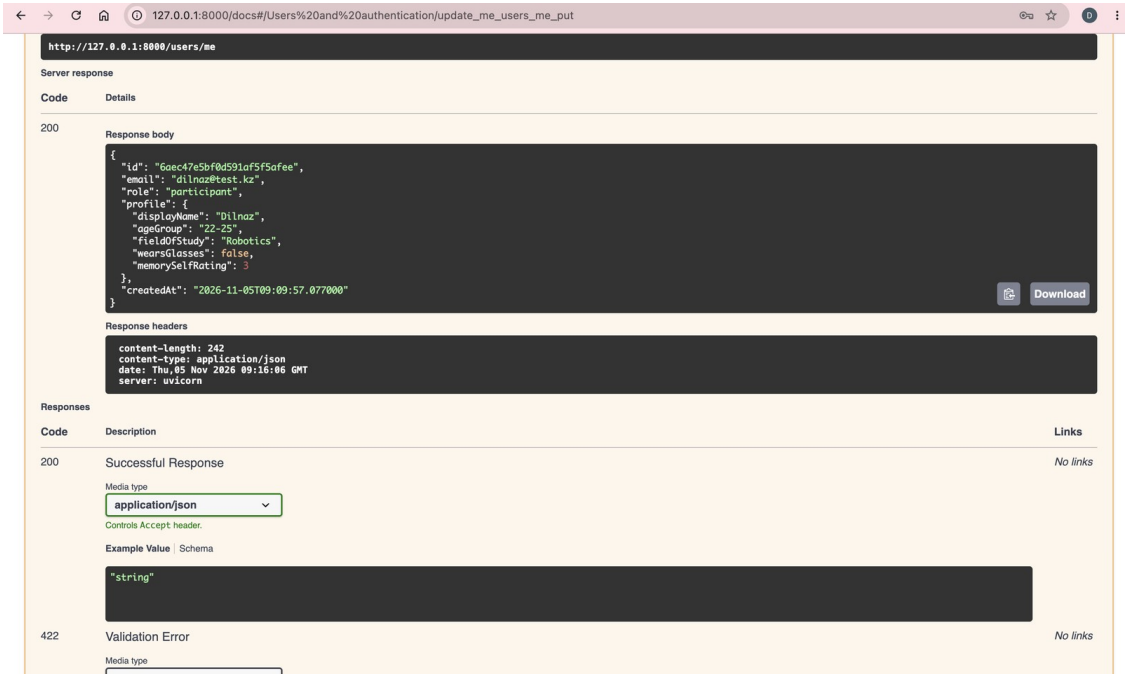
GET /users/me returns the account of the logged-in user. The Curl box shows the header Authorization: Bearer eyJ…, which the server uses to find the user.



PUT /users/me changes only the fields that are sent:

Request:

```
{"profile": {"fieldOfStudy": "Robotics", "memorySelfRating": 3}}
```



Result: fieldOfStudy became "Robotics" and memorySelfRating 3, while displayName, ageGroup and wearsGlasses stayed the same. The code updates the nested fields with `$set` on `profile.fieldOfStudy` and `profile.memorySelfRating`, not the whole profile.

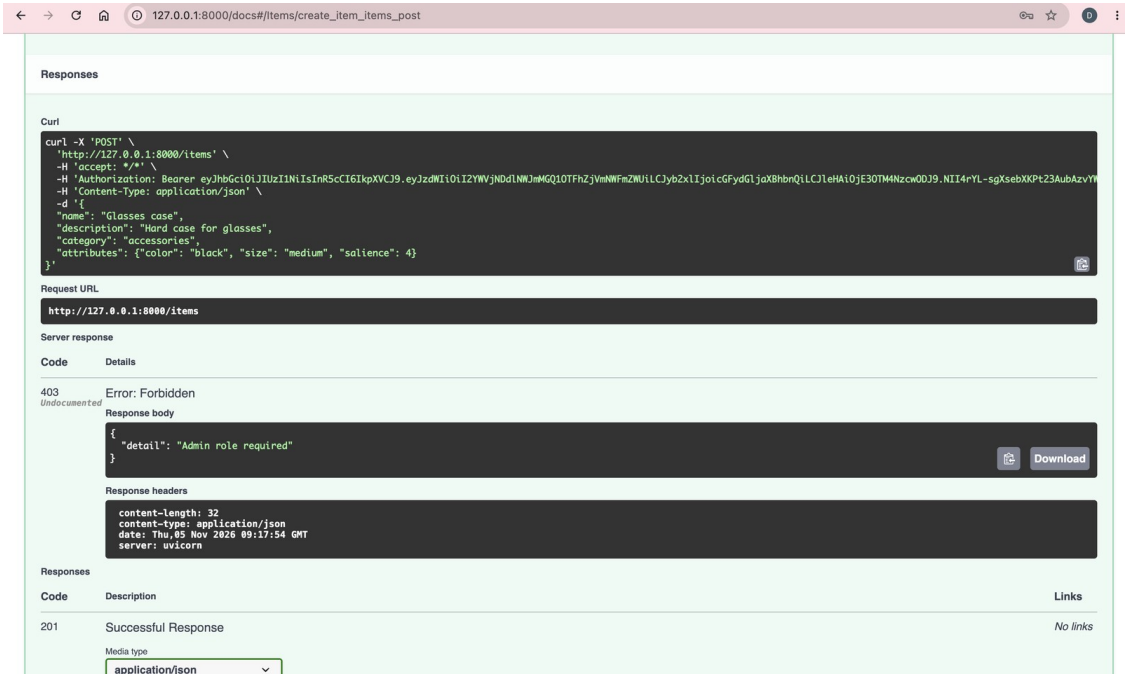
13. Item Catalog

13.1 Access Control:

Only the researcher (role admin) may change the catalog. First I tried to create an item while logged in as a participant:

Request:

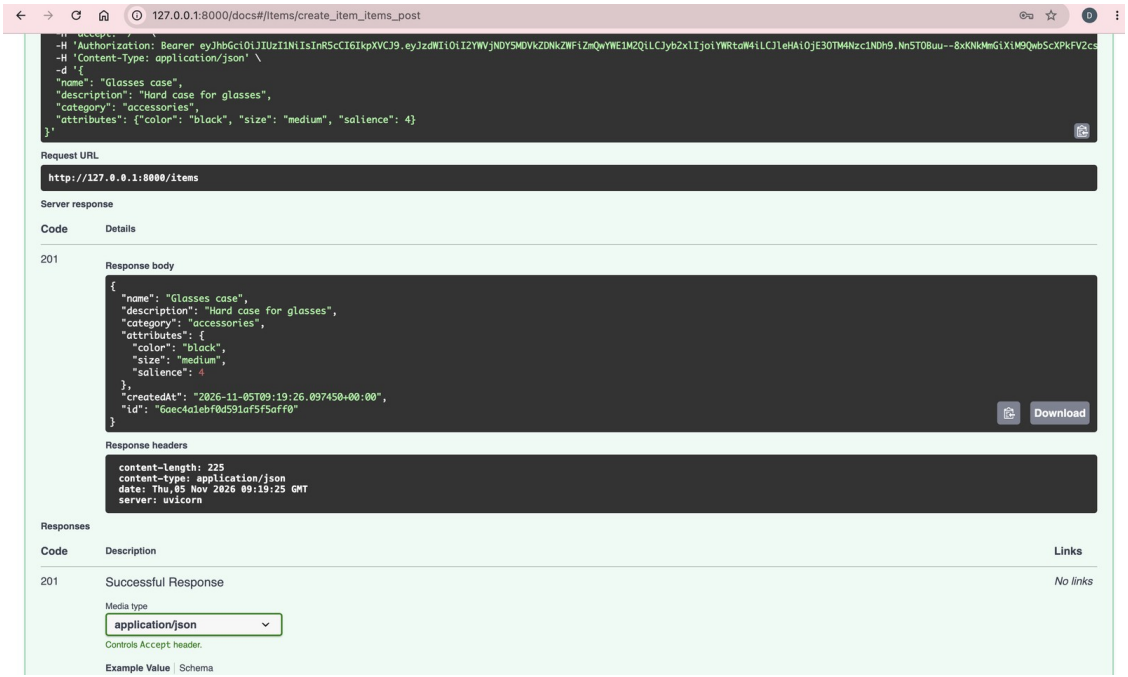
```
{
  "name": "Glasses case",
  "description": "Hard case for glasses",
  "category": "accessories",
  "attributes": {"color": "black", "size": "medium", "salience": 4}
}
```



Result: 403 Forbidden, "Admin role required". The role is read from the token, so a participant cannot change the catalog even with a valid login.

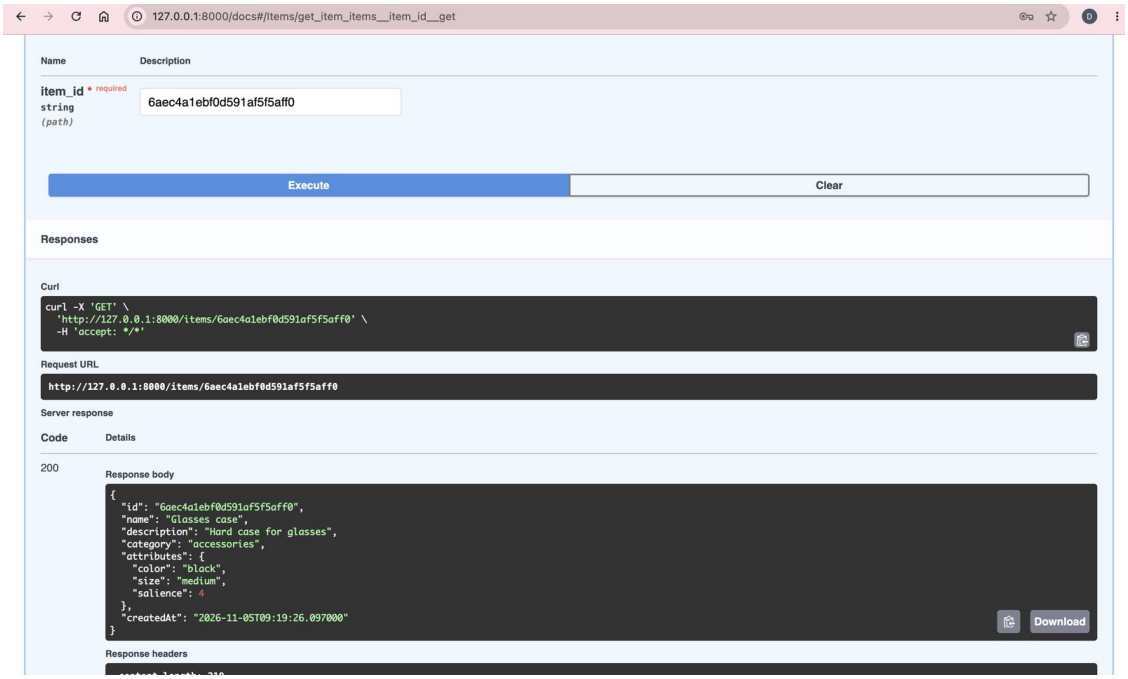
13.2 Create (POST /items):

I logged in as researcher@memoryassist.test and sent the same request.



Result: 201 Created; the item got the id 6aec4a1ebf0d591af5f5aff0.

13.3 Read (GET /items/{item_id}):

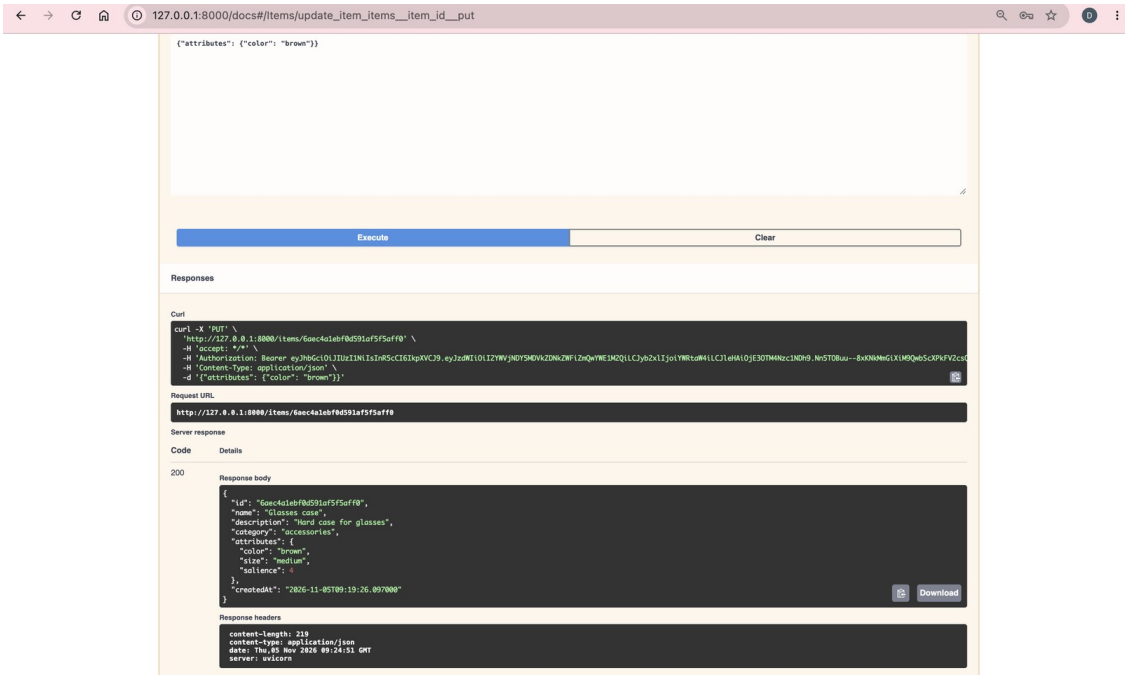


Result: 200 OK with the item. GET /items without an id returns the whole catalog sorted by name, with skip and limit for paging.

13.4 Update (PUT /items/{item_id}):

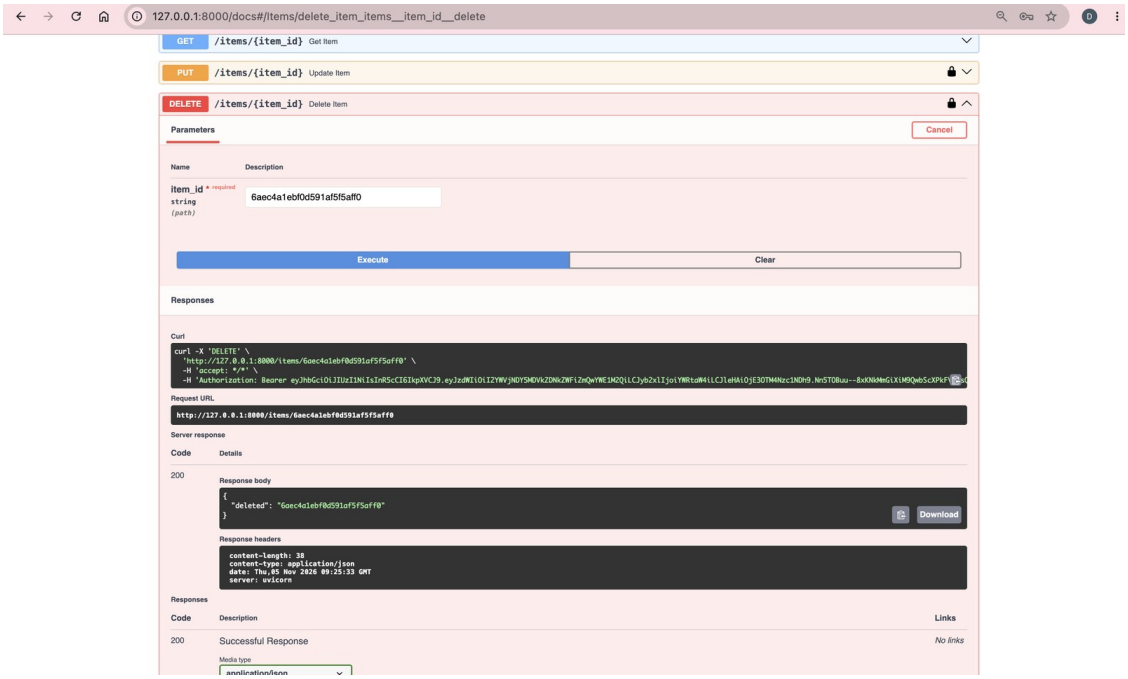
Request:

```
{"attributes": {"color": "brown"}}
```



Result: the colour changed to brown, and size and salience were kept, because only `attributes.color` is updated.

13.5 Delete (DELETE /items/{item_id}):

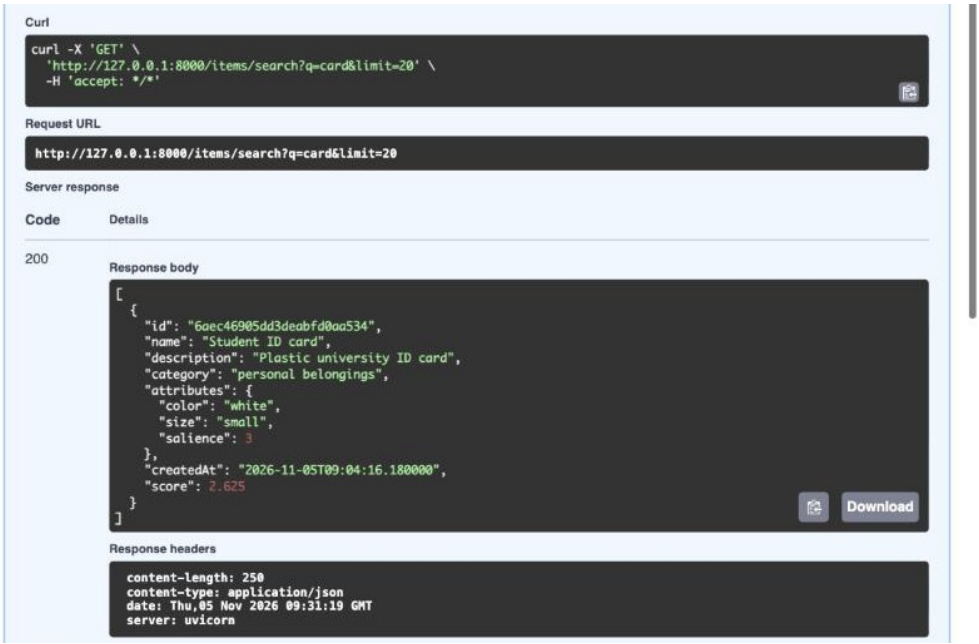


Result: 200 OK, {"deleted": "6aec4a1ebf0d591af5f5aff0"}. An item that already has interactions cannot be deleted (409), so that the history of the experiment is never broken.

14. Search

14.1 Search by Name or Description:

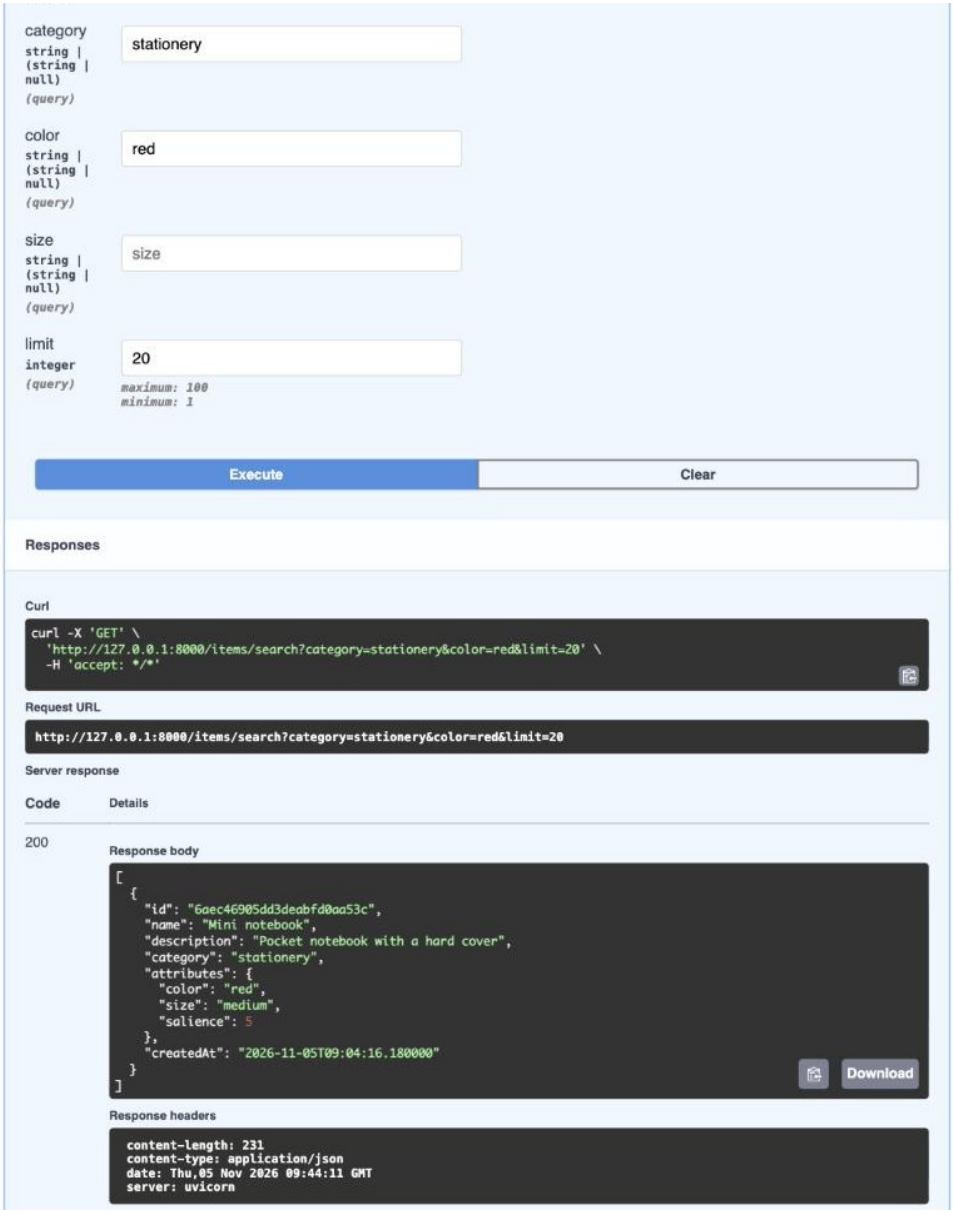
GET /items/search with the parameter q uses the text index. The results are sorted by relevance, which is why each result has a score.



Result: the query q=card found Student ID card.

14.2 Search by Category and Colour:

The same endpoint accepts category, color and size, and any combination of filters. Here category = stationery and color = red:



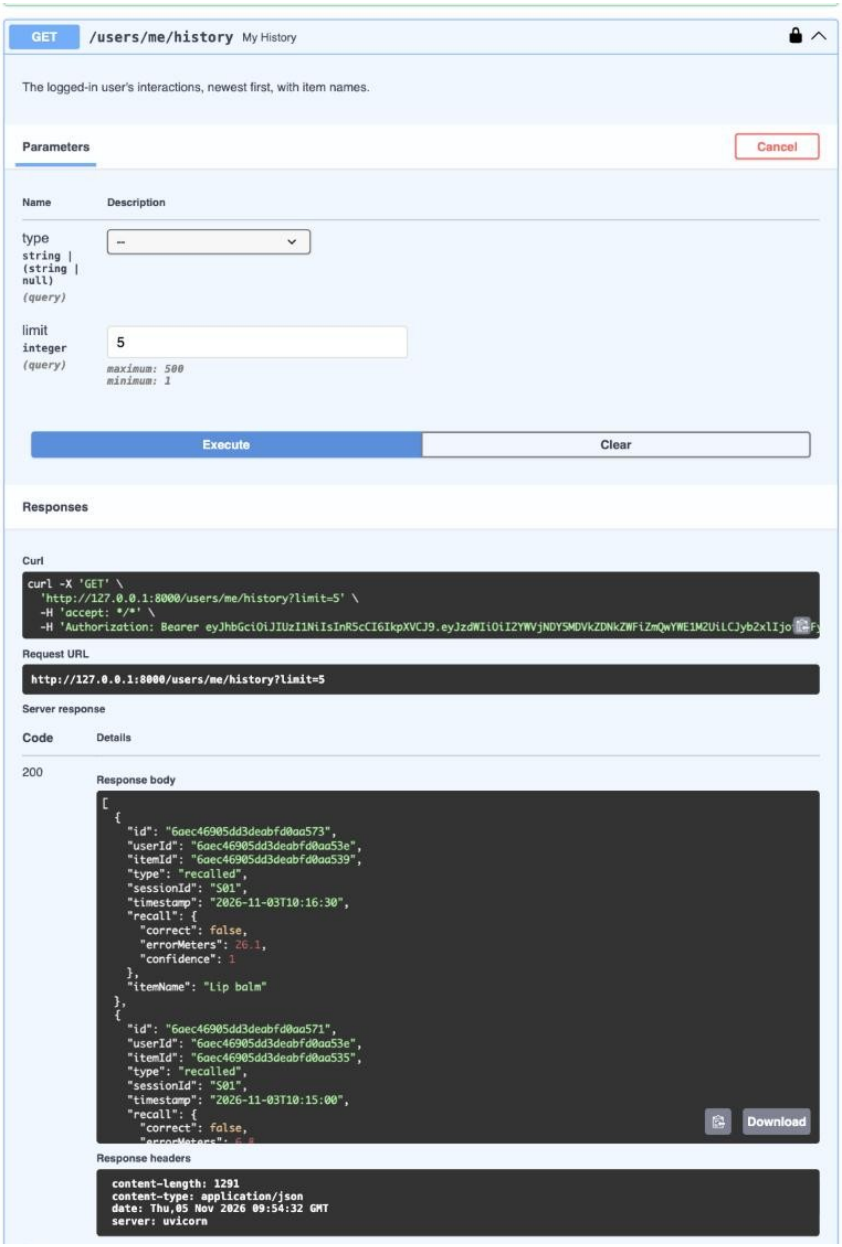
Result: Mini notebook, the only red stationery item. The filters use the indexes on category and attributes.color.

15. User History

For these tests I logged in as participant P01 from the test data (p01@memoryassist.test), whose history was created by the seed script.

15.1 History of the User:

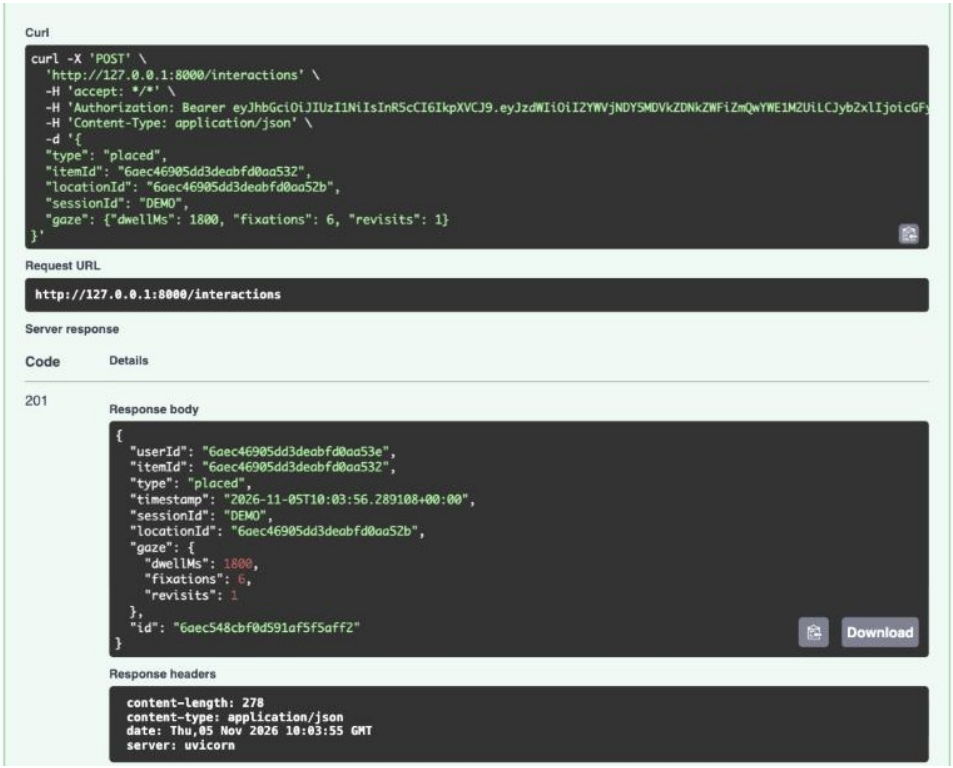
GET /users/me/history returns the interactions of the logged-in user, newest first, with the item name added by a `$lookup` to the items collection. It can be filtered by type.



Result: the last 5 interactions of P01 (limit=5); the first ones are next-day recall results, for example Lip balm was not remembered (correct: false, error 26.1 m, confidence 1).

15.2 Recording an Interaction:

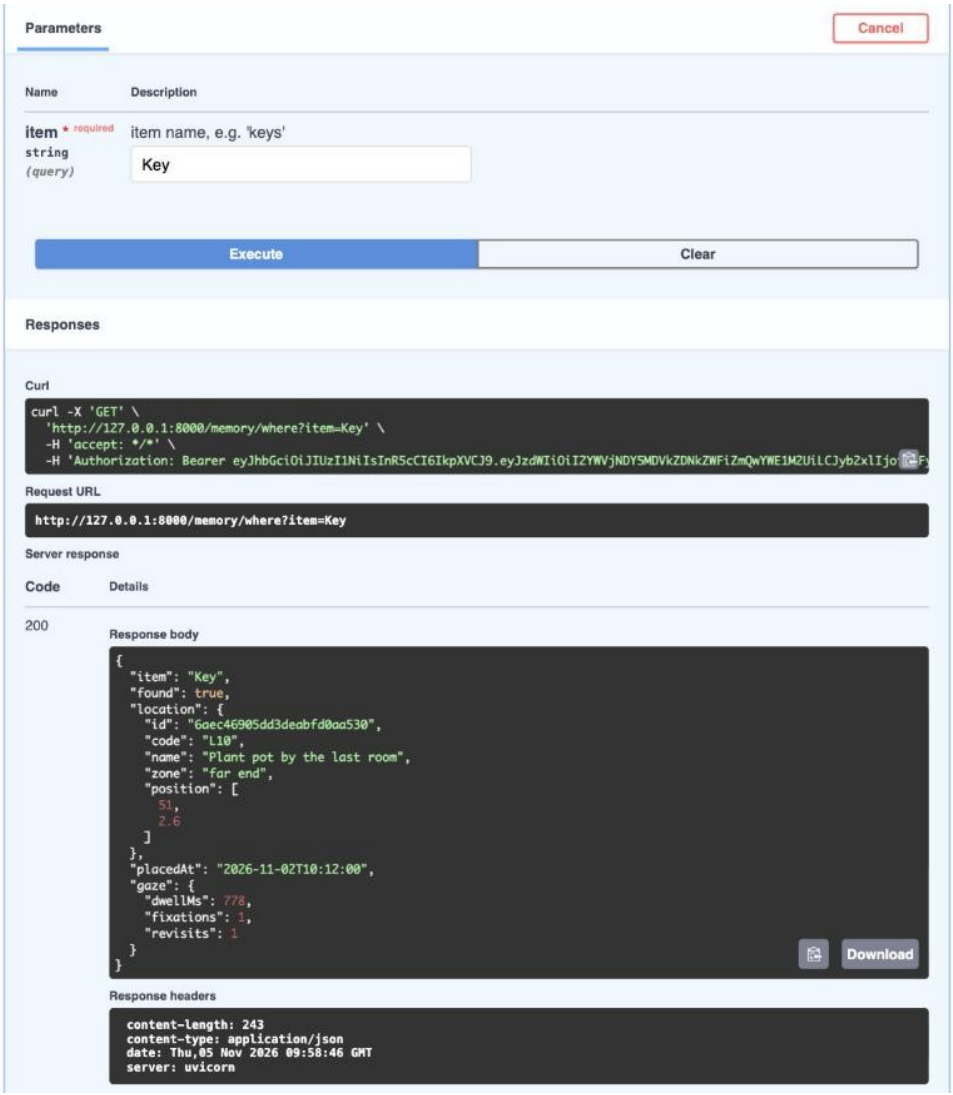
POST /interactions records a new event for the logged-in user. The server checks that the item and location exist and that a "placed" event has gaze data and a "recalled" event has recall data.



Result: 201 Created; P01 placed the Pen at the Printer corner with 1800 ms of gaze, 6 fixations and 1 revisit.

15.3 "Where Did I Leave My …?":

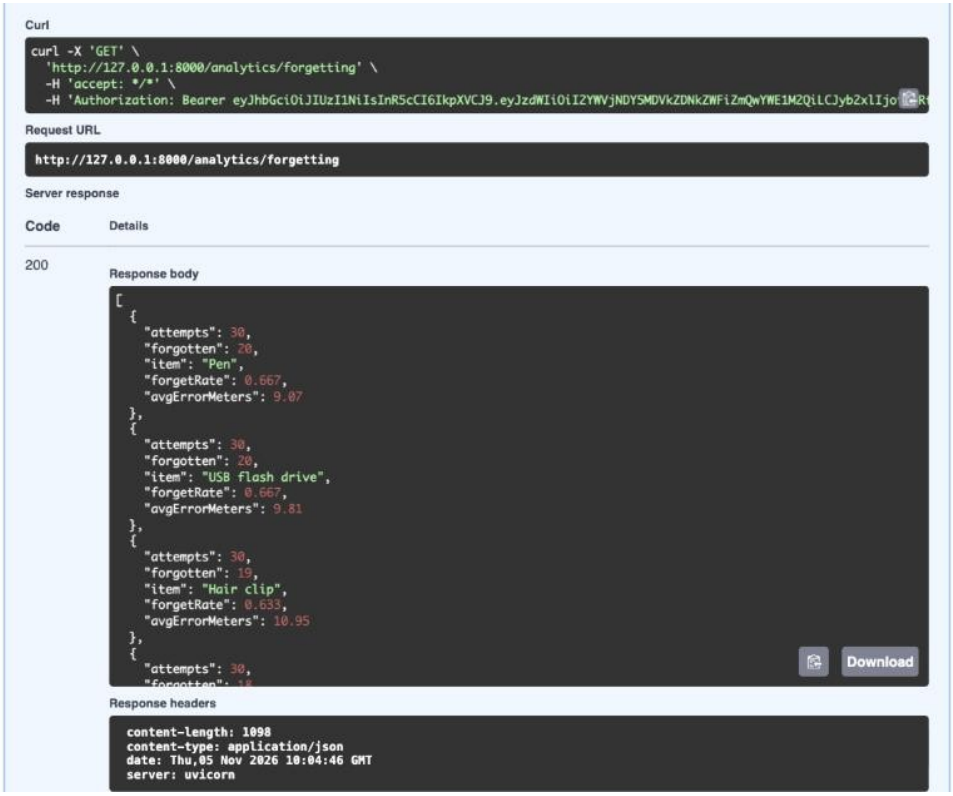
GET /memory/where finds the item by name and returns the last place where this user put it. This is the retrieval function that AR glasses would offer. The question itself is saved as a "queried" interaction.



Result: P01 left the Key at "Plant pot by the last room" (L10, position 51 m, 2.6 m), and looked at it for only 778 ms with one fixation. The query uses the compound index {userId, itemId, type, timestamp}.

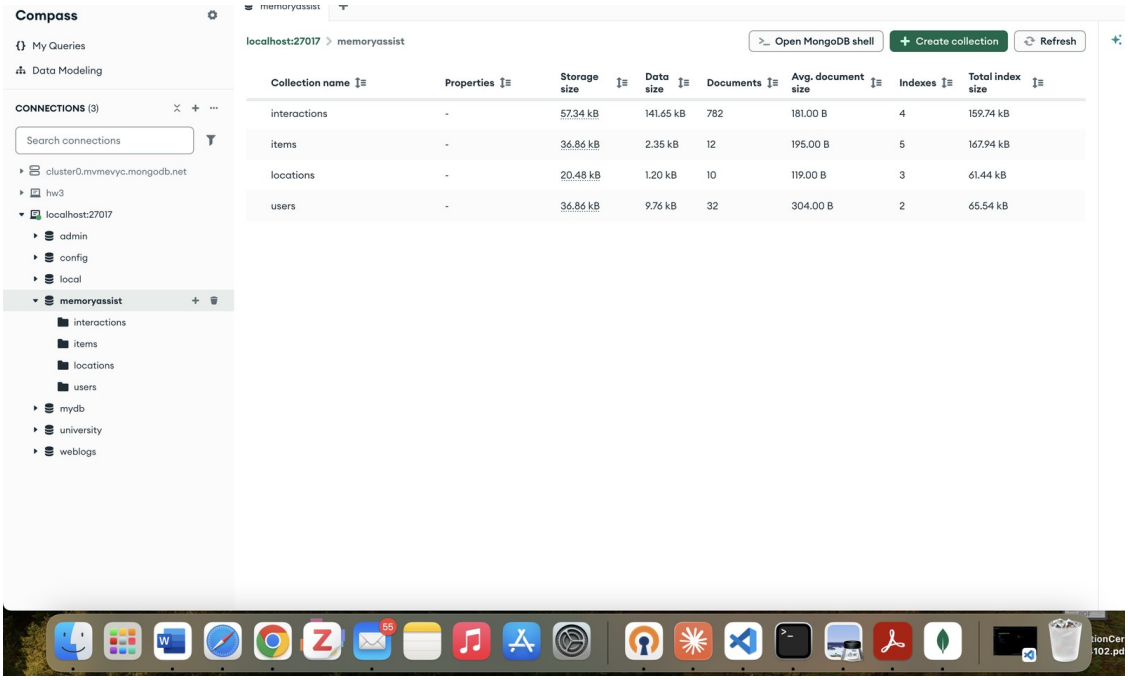
16. Analytics for the Recommender

GET /analytics/forgetting (researcher only) groups all recall results by item with the aggregation framework and calculates the number of attempts, the number of forgotten items, the forget rate and the average map error.



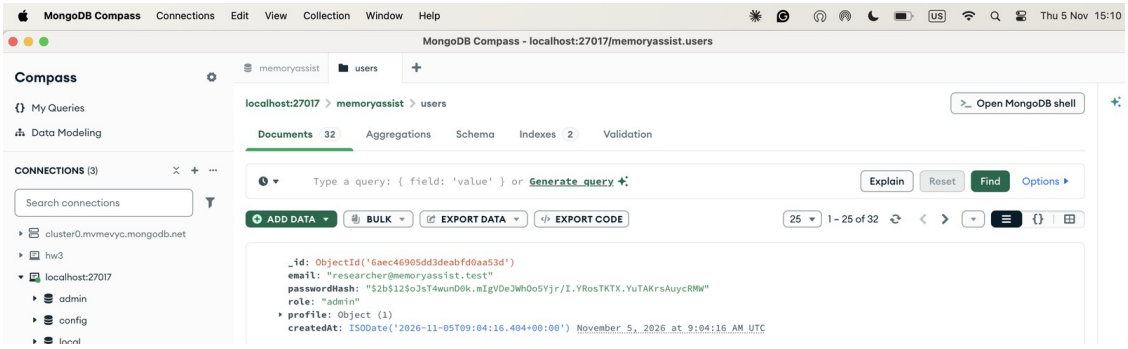
Result: the most forgotten items are the Pen and the USB flash drive (20 of 30, forget rate 0.667) and the Hair clip (0.633); the least forgotten are the Mini notebook (0.233) and the Sticky notes (0.167). The least noticeable items are forgotten most often, as the test data was designed. This per-item forget rate is the first input of the recommender planned for the Final Project.

17. The Database in MongoDB Compass

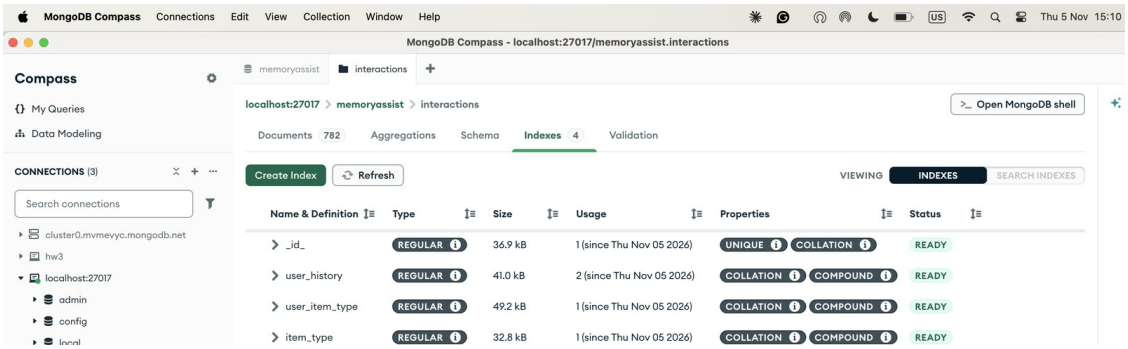


Result: the database memoryassist has the four collections. users has 32 documents (31 from the seed and my test account) and interactions has 782 (780 from the seed plus the "where"

question and the new placement from the tests). The Indexes column counts the indexes created by the code plus the default _id index.



Result: the password is stored only as a bcrypt hash ($2b$12$…), never as plain text.



Result: the indexes of the interactions collection. The Usage column shows that they are really used by the queries: user_history was used by the history requests and user_item_type by the "where" request.

References

Engel, J., Somasundaram, K., Goesele, M., ... Newcombe, R. (2023). Project Aria: A new tool for egocentric multi-modal AI research. arXiv. https://doi.org/10.48550/arXiv.2308.13561

FastAPI. (n.d.). OAuth2 with password (and hashing), Bearer with JWT tokens. FastAPI documentation. Retrieved November 5, 2026, from https://fastapi.tiangolo.com/tutorial/security/oauth2-jwt/

Gurrin, C., Smeaton, A. F., & Doherty, A. R. (2014). LifeLogging: Personal big data. Foundations and Trends in Information Retrieval, 8(1), 1–125. https://doi.org/10.1561/1500000033

Hannula, D. E., Althoff, R. R., Warren, D. E., Riggs, L., Cohen, N. J., & Ryan, J. D. (2010). Worth a glance: Using eye movements to investigate the cognitive neuroscience of memory. Frontiers in Human Neuroscience, 4, 166. https://doi.org/10.3389/fnhum.2010.00166

Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT) (RFC 7519). Internet Engineering Task Force. https://doi.org/10.17487/RFC7519

Koren, Y., Bell, R., & Volinsky, C. (2009). Matrix factorization techniques for recommender systems. Computer, 42(8), 30–37. https://doi.org/10.1109/MC.2009.263

MongoDB, Inc. (n.d.-a). $lookup (aggregation). MongoDB Manual. Retrieved November 5, 2026, from https://www.mongodb.com/docs/manual/reference/operator/aggregation/lookup/

MongoDB, Inc. (n.d.-b). Compound indexes. MongoDB Manual. Retrieved November 5, 2026, from https://www.mongodb.com/docs/manual/core/indexes/index-types/index-compound/

MongoDB, Inc. (n.d.-c). Data modeling. MongoDB Manual. Retrieved November 5, 2026, from https://www.mongodb.com/docs/manual/data-modeling/

MongoDB, Inc. (n.d.-d). Indexes. MongoDB Manual. Retrieved November 5, 2026, from https://www.mongodb.com/docs/manual/indexes/

MongoDB, Inc. (n.d.-e). Text indexes. MongoDB Manual. Retrieved November 5, 2026, from https://www.mongodb.com/docs/manual/core/indexes/index-types/index-text/

Neo4j, Inc. (n.d.). Neo4j Graph Data Science Library Manual. Retrieved November 5, 2026, from https://neo4j.com/docs/graph-data-science/current/

OWASP Foundation. (n.d.). Password storage cheat sheet. OWASP Cheat Sheet Series. Retrieved November 5, 2026, from https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

Provos, N., & Mazières, D. (1999). A future-adaptable password scheme. In Proceedings of the 1999 USENIX Annual Technical Conference (pp. 81–91). USENIX Association.

Redis Ltd. (n.d.). Redis documentation. Retrieved November 5, 2026, from https://redis.io/docs/latest/

Sarwar, B., Karypis, G., Konstan, J., & Riedl, J. (2001). Item-based collaborative filtering recommendation algorithms. In Proceedings of the 10th International Conference on World Wide Web (pp. 285–295). ACM. https://doi.org/10.1145/371920.372071